

Data Validation

Pandas vs. Great Expectations

Agenda

- 1 What is Data Validation & Why It Matters
- 2 The Dataset: users.csv — Structure & Injected Errors
- 3 Version 1: Pure Pandas Validation (DataValidator Class)
- 4 Validation Dimensions: Schema, Completeness, Ranges, Uniqueness, Categorical
- 5 Generating & Reading the Pandas Report
- 6 Version 2: Great Expectations v1.x Modern API
- 7 GX Core Concepts: Context, Suite, Batch, Validation Definition
- 8 Running & Reading the GX Report & Data Docs
- 9 Pandas vs Great Expectations — Comparison

Section 1: What is Data Validation & Why It Matters

What is Data Validation?

Data validation is the process of checking **whether data conforms to defined rules**, constraints, and quality standards before it is used for analysis, modeling, or reporting.

Accuracy	Completeness	Uniqueness	Consistency
Values are correct and realistic (e.g., age ≥ 0)	No critical missing (NULL) values	No duplicate records in key columns	Schema & categories match expectations

Why validate?

- Garbage In = Garbage Out — bad data ruins ML models and reports
- Catch errors early when they are cheap to fix
- Build trust in datasets shared across teams

The 6 Dimensions of Data Quality

Accuracy	Values are within valid business ranges	e.g. age: 0-120, user_id $\geq$ 1
Completeness	No NULLs in required columns	e.g. user_id, email, name must exist
Uniqueness	Key columns have no duplicates	e.g. user_id, email must be unique
Consistency	Schema columns & types match spec	e.g. columns list matches, types correct
Validity	Values match allowed sets or patterns	e.g. country in ['USA', 'UK', ...], email $\sim$.+@.+
Distribution	Statistical properties are as expected	e.g. mean(age) in [30,60], row count 100-100k



These 6 dimensions map directly to both the Pandas checks and Great Expectations rules in our files.

Section 2: The Dataset — users.csv

generate_users_data.py — Creating the Dataset

File: generate_users_data.py

```
import pandas as pd
import numpy as np

np.random.seed(42)    # ensures reproducible random data
n = 200                # total number of user rows

df = pd.DataFrame({
    'user_id': range(1, n + 1),                # IDs 1-200
    'name':    ['User_' + str(i) for i in range(1,n+1)],
    'email':    ['user' + str(i) + '@email.com' for i in range(1,n+1)],
    'age':      np.random.randint(18, 80, n),   # high=80 exclusive → 18-79
    'signup_date': pd.date_range('2022-01-01', periods=n, freq='D'),
    'country': np.random.choice(['USA', 'UK', 'Canada', 'Australia', 'Germany'], n)
})
```



np.random.seed(42) — '42' is arbitrary; any integer gives the same reproducible sequence.



Pro Tip: Always set a random seed when generating synthetic data for teaching or testing.

Intentional Data Errors — What We Inject & Why

Four intentional errors are injected to test the validators:

```
df.loc[5, 'email'] = None    # missing email    → tests Completeness check
df.loc[10, 'age'] = -5       # invalid age       → tests Range check (age ≥ 0)
df.loc[15, 'country'] = 'Egypt' # invalid country → tests Categorical check
df.loc[1, 'user_id'] = 1     # duplicate ID     → tests Uniqueness check
```

`.loc[row_label, col_label]` explained:

- `df.loc[5, 'email'] = None` → row at index label 5, column 'email', set to NaN/None
- Index labels (5, 10, 15, 1) are NOT row numbers by default — they match the index value
- Since we used `range(1, n+1)` for `user_id` AND it's the default index, `df.loc[1]` = row with index 1

Save & Reload:

```
df.to_csv('users.csv', index=False) # index=False: don't write row numbers as a column
df = pd.read_csv('users.csv', parse_dates=['signup_date']) # parse_dates restores datetime
type
```


users.csv — Schema & Sample Data

Column Schema

Column	Type	Example	Notes
user_id	int64	1, 2, 3...	Primary key, must be unique ≥ 1
name	object	User_1	String label
email	object	user1@email.com	Must match <code>.+@.+</code>
age	int64	34	Valid range: 0–120
signup_date	datetime64	2022-01-01	Parsed from string
country	object	USA	One of 5 allowed values



After `read_csv` without `parse_dates`, `signup_date` would be 'object' (string) — always parse dates explicitly.

Section 3: Version 1 — Pure Pandas Validation

DataValidator Class — Architecture Overview

Class structure in generate_report_python.py:

```
class DataValidator:
```

```
    __init__
```

```
Initialize self.validation_results = []
```

```
    validate_schema(df, expected_columns, expected_types)
```

```
Check columns exist & dtypes match
```

```
    validate_completeness(df, required_columns,  
                           max_missing_pct)
```

```
Check % of NULLs ≤ threshold
```

```
    validate_ranges(df, range_rules)
```

```
Check min/max bounds for numeric cols
```

```
    validate_uniqueness(df, unique_columns)
```

```
Check for duplicate values in key cols
```

```
    validate_categorical(df, categorical_rules)
```

```
Check values are in allowed set
```

```
    generate_report()
```

```
Print summary & return dict with stats
```

How each validation method works:

- 1. Create a report dict with timestamp, check_type, passed=True, issues=[]
- 2. Apply pandas checks (column comparison, isnull, duplicated, etc.)
- 3. If a check fails → set passed=False → append message to issues list
- 4. Append the report to self.validation_results
- 5. Return the report dict (also stored inside the object)



Each validate_*() method is independent — you can run any subset of checks.

validate_schema() — Checking Columns & Data Types

```
def validate_schema(self, df, expected_columns, expected_types):  
    # Build a report skeleton with metadata  
    report = { 'check_type': 'Schema', 'passed': True, 'issues': [] }  
  
    # 1. Find any columns present in spec but MISSING from df  
    missing_cols = set(expected_columns) - set(df.columns)  
    if missing_cols:  
        report['passed'] = False  
        report['issues'].append(f'Missing columns: {missing_cols}')  
  
    # 2. Check that each column's dtype matches what we expect  
    for col, expected_type in expected_types.items():  
        if col in df.columns:  
            actual_type = str(df[col].dtype) # e.g. 'int64', 'object'  
            if actual_type != expected_type:  
                report['issues'].append(f'Column {col}: expected {expected_type}, got {actual_type}')
```

Arguments: `df` = pandas DataFrame | `expected_columns` = list of column names | `expected_types` = dict {col: dtype_str}

Return: dict with keys: check_type, passed (bool), issues (list), **timestamp**

💡 `str(df[col].dtype)` converts numpy dtype object to string like 'int64'. This is needed for string comparison.

validate_completeness() — Checking for Missing Values

```
def validate_completeness(self, df, required_columns, max_missing_pct=0.05):
    # max_missing_pct=0.05 → default: allow at most 5% missing
    report = { 'check_type': 'Completeness', 'passed': True, 'issues': [] }

    for col in required_columns:
        missing_count = df[col].isnull().sum()          # count of True (NaN) values
        missing_pct   = (missing_count / len(df)) * 100 # convert to percent

        if missing_pct > max_missing_pct * 100:        # compare in same unit (pct)
            report['passed'] = False
            report['issues'].append(
                f'Column {col}: {missing_pct:.2f}% missing (allowed: {max_missing_pct*100:.0f}%)'
            )
    self.validation_results.append(report)
    return report
```

Key pandas methods:

`df[col].isnull()` → boolean Series (True where NaN/None) | `.sum()` → count of True (i.e. missing) |
`len(df)` → total rows

💡 In our data: `df.loc[5,'email']=None` injects 1 missing → $1/200 = 0.5\%$, allowed 1%

validate_ranges() — Checking Min/Max Bounds

```
def validate_ranges(self, df, range_rules):
    # range_rules = { 'age': {'min': 0, 'max': 120}, 'user_id': {'min': 1} }
    report = { 'check_type': 'Ranges', 'passed': True, 'issues': [] }

    for col, rules in range_rules.items():
        if col not in df.columns:
            continue                # skip if column doesn't exist

        if 'min' in rules:
            below_min = df[df[col] < rules['min']] # boolean filter
            if len(below_min) > 0:
                report['passed'] = False
                report['issues'].append(
                    f'Column {col}: {len(below_min)} values below minimum ({rules["min"]})'
                )
```

df[df[col] < rules['min']] — Boolean Indexing Explained:

- `df[col] < rules['min']` → boolean Series like `[False, False, True, ...]`
- `df[boolean_series]` → filters to only rows where condition is True (rows that violate min)
- `len(below_min) > 0` → if any violations found → fail the check

validate_uniqueness() & validate_categorical()

validate_uniqueness() — no duplicate values in key columns:

```
duplicate_count = df[col].duplicated().sum()  
# .duplicated() → True for ALL duplicates EXCEPT the first occurrence  
# .sum()          → count of duplicate rows found
```

validate_categorical() — values must be in an allowed set:

```
actual_values = set(df[col].dropna().unique())  
# .dropna() → remove NaN before checking (NaN has no category)  
# .unique() → array of distinct values  
# set(...) → convert to Python set for fast set arithmetic  
  
invalid_values = actual_values - set(allowed_values)  
# Set difference: values present in data but NOT in allowed list  
# e.g. {'Egypt'} - {'USA', 'UK', 'Canada', 'Australia', 'Germany'} = {'Egypt'}
```

💡 .dropna() is important — without it, NaN would become a 'value' and trigger a false invalid-category warning.

★ Pro Tip: Use set arithmetic (A - B) instead of loops for finding invalid values — it's faster and more readable.

generate_report() — Printing the Validation Summary

```
def generate_report(self):
    total = len(self.validation_results)
    passed = sum(1 for r in self.validation_results if r['passed'])
    failed = total - passed
    success_rate = (passed / total * 100) if total > 0 else 0

    for result in self.validation_results:
        status = 'PASS' if result['passed'] else 'FAIL'
        print(f'[{status}] {result["check_type"]} Check')
        for issue in result['issues']:
            print(f'    {issue}')          # indent issues for readability

    return {'total': total, 'passed': passed, 'failed': failed,
            'success_rate': success_rate, 'details': self.validation_results}
```

Return value — dict with keys:

total (int) | **passed** (int) | **failed** (int) | **success_rate** (float, %) | **details** (list of report dicts)



sum(1 for r in ... if r['passed']) uses a generator expression — memory-efficient way to count conditionally.

Understanding the Report Output Structure

Each validation method returns a report dict:

```
{
  'timestamp': '2024-01-15 10:30:00',    # when the check ran
  'check_type': 'Completeness',          # which validator method
  'passed': False,                        # True if no violations found
  'issues': [                             # list of problem descriptions
    "Column 'email': 0.50% missing (allowed max: 1%)"
  ]
}
```

generate_report() aggregates all results:

```
{
  'total': 5,      # number of validate_*() methods called
  'passed': 1,     # number that passed (Schema passes in our example)
  'failed': 4,     # Completeness, Ranges, Uniqueness, Categorical fail
  'success_rate': 20.0, # 1/5 * 100
  'details': [...], # list of all individual report dicts
}
```

Defining Validation Rules — Input Dictionaries

1. Expected column names (list)

```
expected_columns = ['user_id', 'name', 'email', 'age', 'signup_date', 'country']
```

2. Expected data types (dict: column → dtype string)

```
expected_types = {  
    'user_id': 'int64',  
    'name': 'str',          # old versions: pandas 'object' = Python string  
    'email': 'str',  
    'age': 'int64',  
    'signup_date': 'datetime64[us]', # [ns] = nanosecond precision  
    'country': 'str'  
}
```

3. Range rules (dict: column → {min, max})

```
range_rules = { 'age': {'min': 0, 'max': 120}, 'user_id': {'min': 1} }
```

4. Categorical rules (dict: column → list of allowed values)

```
categorical_rules = { 'country': ['USA', 'UK', 'Canada', 'Australia', 'Germany'] }
```

Running the Pandas Validator — Full Workflow

1. Instantiate the validator

```
validator = DataValidator()
```

2. Run each check (order doesn't matter – all results are stored)

```
validator.validate_schema(df, expected_columns, expected_types)
validator.validate_completeness(df, required_columns=['user_id', 'email'],
max_missing_pct=0.01)
validator.validate_ranges(df, range_rules)
validator.validate_uniqueness(df, unique_columns=['user_id', 'email'])
validator.validate_categorical(df, categorical_rules)
```

3. Print report and capture return value

```
summary = validator.generate_report()
print(f'Success Rate: {summary["success_rate"]:.1f}%')
```

Expected output (truncated):

```
[FAIL] Ranges      → Column 'age': 1 values below minimum (0)
[FAIL] Uniqueness  → Column 'user_id': 1 duplicate values found
[FAIL] Categorical → Column 'country': invalid values: {'Egypt'}
```

Pandas Validator — Tips, Pitfalls & Pros/Cons

Pros

- Zero extra dependencies — pandas only
- Easy to understand & customize for beginners
- Full Python control — add any custom logic

Cons

- No built-in reporting / data docs
- Manual maintenance — rules are free-form dicts
- No standard vocabulary — harder to share with teams

Common Pitfalls & Solutions

X

Date types lost after CSV round-trip

→ Always use `parse_dates=['signup_date']` in `read_csv()`

X

`isnull()` vs `isna()`

→ They are identical aliases — use either, be consistent

X

Off-by-one in `.loc` vs `.iloc`

→ `.loc` uses index `labels`; `.iloc` uses `integer` position

Section 4: Version 2 — Great Expectations v1.x

What is Great Expectations (GX)?

Great Expectations is the most popular open-source Python framework for **data validation**. It lets you **describe what you expect your data** to look like using a rich library of **built-in expectations**.

Install:

```
pip install great-expectations pandas
```

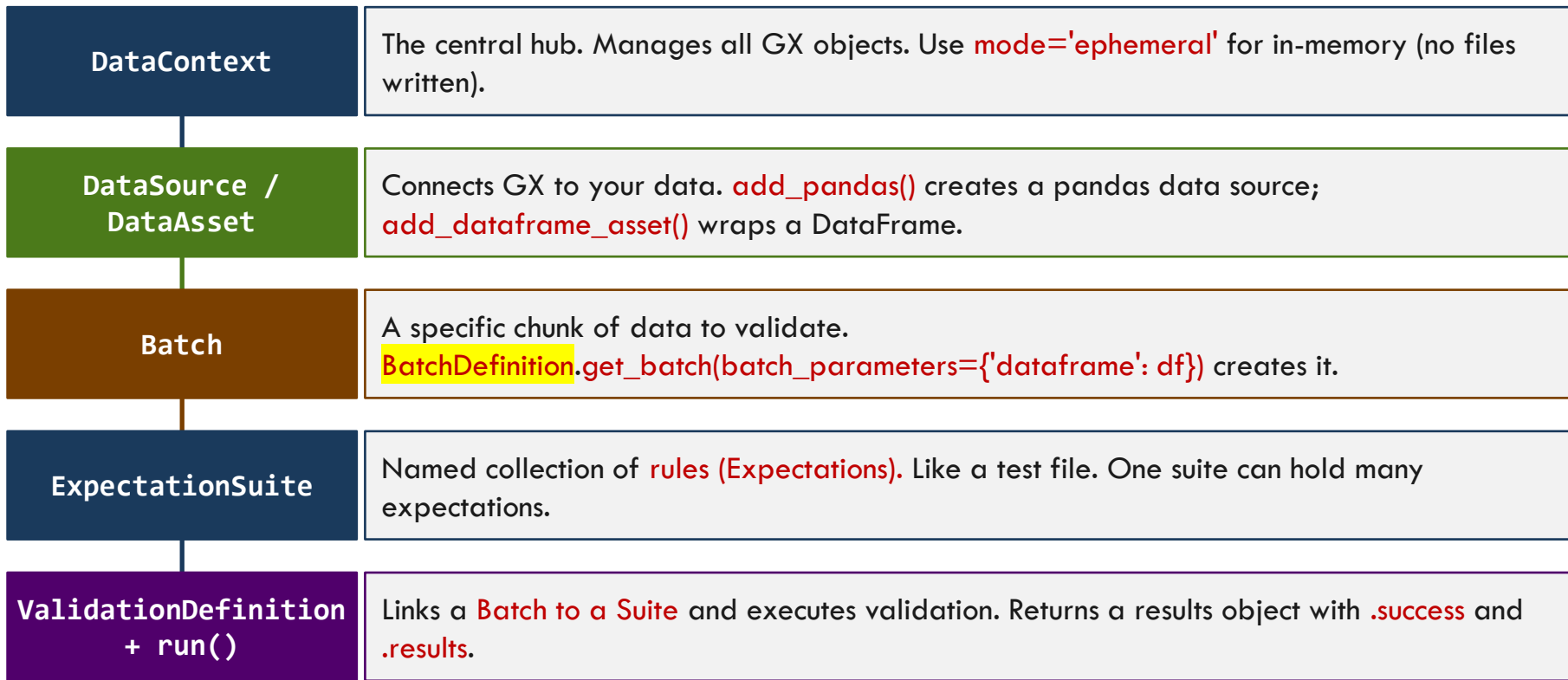
Key features:

- 100+ built-in Expectations (range, null, regex, set, stats...)
- Auto-generated HTML Data Docs — shareable validation reports
- Version 1.x has a cleaner, modern API vs older 0.x versions
- Works with pandas, Spark, SQL databases, and more
- Integrates with dbt, Airflow, and other data pipeline tools



GX v1.x API is significantly different from 0.x — if you find old tutorials, check which version they use.

GX v1.x — 5 Core Concepts You Must Know



GX Context mode='file' — Persistent File-Based Context

Persist suites, results & Data Docs to disk for production projects

mode='ephemeral' — in-memory only (no files saved)

```
context = gx.get_context(mode='ephemeral')
```

mode='file' — persists config/results to disk

```
context = gx.get_context(mode='file', project_root_dir='./gx_project')
```

With mode='file', GX creates this folder structure:

```
# gx_project/
```

```
# |— gx/
```

```
# |   |— great_expectations.yml      # main config file
```

```
# |   |— expectations/              # saved Expectation Suites (JSON)
```

```
# |       |— users_suite.json
```

```
# |   |— checkpoints/              # saved Checkpoint configs
```

```
# |   |— plugins/                  # custom plugins
```

```
# |   |— uncommitted/
```

```
# |       |— data_docs/            # HTML validation reports
```

```
# |           |— local_site/
```

```
# |               |— index.html
```

```
# |       |— validations/          # validation result JSON files
```

```
# |           |— users_suite/
```

```
# |               |— run_20240115/
```

```
# |                   |— result.json
```

After context.build_data_docs() you can open:

```
# gx_project/gx/uncommitted/data_docs/local_site/index.html
```

💡 Use mode='ephemeral' for quick scripts. Use mode='file' in real projects to persist suites, track history, and share HTML Data Docs with your team.

GX Steps 1 & 2 — Context, DataSource & Batch

Steps 1 & 2: Connect GX to your DataFrame

```
import great_expectations as gx

# Step 1: Create an in-memory GX context (no config files written to disk)
context = gx.get_context(mode='ephemeral')

# Step 2a: Add a pandas DataSource (tells GX we're using pandas)
data_source = context.data_sources.add_pandas(name='my_pandas_source')

# Step 2b: Wrap your DataFrame as a named 'asset'
data_asset = data_source.add_dataframe_asset(name='users_asset')

# Step 2c: Define how to slice the data (whole DataFrame = one batch)
batch_def = data_asset.add_batch_definition_whole_dataframe('my_batch')

# Step 2d: Get the actual batch (passing the DataFrame here)
batch = batch_def.get_batch(batch_parameters={'dataframe': df})
```



mode='ephemeral' means GX won't create any folders or config files. Perfect for scripts and notebooks.

GX v1.13 — Other Data Sources (alternatives to add_pandas)

context.data_sources.<function>(name='my_source', ...)

Function	Data Source Type	Example Use Case
add_pandas()	Pandas DataFrame	In-memory DataFrames (default in demos)
add_spark()	Apache Spark DataFrame	Big data / PySpark pipelines on clusters
add_postgres()	PostgreSQL database	Relational DBs with psycopg2 connection
add_mysql()	MySQL database	MySQL/MariaDB with SQLAlchemy connection
add_sqlite()	SQLite database	Lightweight file-based SQL databases
add_snowflake()	Snowflake data warehouse	Cloud DW; needs snowflake-sqlalchemy
add_databricks_sql()	Databricks SQL	Databricks SQL endpoints / Unity Catalog
add_bigquery()	Google BigQuery	GCP analytical warehouse
add_redshift()	Amazon Redshift	AWS cloud data warehouse
add_mssql()	Microsoft SQL Server	Enterprise MS SQL databases
add_pandas_filesystem()	Files on filesystem (CSV, Parquet...)	Batch files; reads with Pandas engine

GX v1.13 — Asset Types (alternatives to `add_dataframe_asset`)

`data_source.<function>(name='my_asset', ...)`

Function (on DataSource)	Asset Type	Use When
<code>add_dataframe_asset()</code>	In-memory DataFrame	Pandas / Spark DF already loaded in RAM
<code>add_csv_asset()</code>	CSV file	Validate a single CSV file from disk or URL
<code>add_parquet_asset()</code>	Parquet file	Validate Parquet columnar files
<code>add_json_asset()</code>	JSON file	Validate JSON-formatted data files
<code>add_excel_asset()</code>	Excel file (.xlsx)	Validate Excel spreadsheet data
<code>add_table_asset()</code>	Database table	Validate a full SQL table (Postgres, MySQL...)
<code>add_query_asset()</code>	SQL query result	Validate results of a custom SELECT query
<code>add_directory_csv_asset()</code>	Folder of CSV files	Batch-validate multiple CSV files at once



The asset type must match the data source type. **Pandas sources support file-based assets**; SQL sources support table/query assets.

GX v1.13 — Batch Definition Alternatives

`data_asset.<function>(name='my_batch_def', ...)` — controls HOW data is split into batches

Function (on DataSet)	Batch Strategy	Use When
<code>add_batch_definition_whole_dataframe()</code>	Entire DataFrame as one batch	Default — validate all data at once
<code>add_batch_definition_whole_table()</code>	Entire SQL table as one batch	SQL table assets; no partitioning needed
<code>add_batch_definition_daily()</code>	One batch per calendar day	Time-partitioned data; validate by day
<code>add_batch_definition_monthly()</code>	One batch per calendar month	Monthly partitioned data or reports
<code>add_batch_definition_yearly()</code>	One batch per calendar year	Annual datasets or long-term data
<code>add_batch_definition_by_regex()</code>	Batches matched by regex pattern	Multiple files named by date/ID pattern
<code>add_batch_definition_by_column()</code>	Batches partitioned by column value	SQL assets; split by a partition column

💡 For DataFrames: only `add_batch_definition_whole_dataframe()` applies — DataFrames are always passed in full. Partitioned batch defs (daily/monthly/regex) apply to file-based and SQL assets.

GX Step 3 — Creating an ExpectationSuite

```
# Step 3: Create a named Expectation Suite (collection of rules)
suite = context.suites.add(
    gx.ExpectationSuite(name='users_validation_suite')
)

# Add individual expectations to the suite:

# ACCURACY – age must be between 0 and 120
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToBeBetween(
        column='age', min_value=0, max_value=120
    )
)

# ACCURACY – email must match regex pattern
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToMatchRegex(
        column='email', regex=r'.+@.+' # at least one char, @, at least one char
    )
)
```



ExpectationSuite(name=...) — the name is used to identify the suite in reports and Data Docs.

All 9 Expectations Used — Reference Table

Expectation Class	Dimension	What it checks
ExpectColumnValuesToBeBetween	Accuracy	Values in [min_value, max_value] range
ExpectColumnValuesToMatchRegex	Accuracy	Each value matches the regex pattern
ExpectColumnValuesToNotBeNull	Completeness	Column has no NULL / NaN values
ExpectColumnValuesToBeUnique	Uniqueness	No duplicate values in column
ExpectTableColumnsToMatchSet	Consistency	Table has exactly the listed columns
ExpectColumnValuesToBeInSet	Validity	Values are members of allowed set
ExpectColumnMeanToBeBetween	Distribution	Column mean falls in [min, max]
ExpectTableRowCountToBeBetween	Distribution	Row count within [min_value, max_value]



Great Expectations has 100+ built-in expectations. Browse them at greatexpectations.io/docs.

GX Expectations — Completeness (Pandas)

Ensure required data is present and not missing/null

Expectation Method	Example Argument(s)	Validates What
ExpectColumnValuesToNotBeNull	column='email'	Column has no NULL/NaN values at all
ExpectColumnValuesToBeNull	column='deleted_at', mostly=0.9	Column IS null (e.g. soft-delete flag mostly unset)

GX Expectations — Numeric (Pandas)

Validate numeric ranges, statistics and distributions

Expectation Method	Example Argument(s)	Validates What
ExpectColumnValuesToBeBetween	column='age', min_value=0, max_value=120	Values are within numeric min/max bounds
ExpectColumnMeanToBeBetween	column='age', min_value=18, max_value=65	Column mean falls in expected range
ExpectColumnMedianToBeBetween	column='score', min_value=40, max_value=90	Column median is within expected range
ExpectColumnSumToBeBetween	column='revenue', min_value=0	Column sum is within bounds (non-negative)
ExpectColumnMinToBeBetween	column='age', min_value=0, max_value=18	Column minimum value is in acceptable range
ExpectColumnMaxToBeBetween	column='age', min_value=100, max_value=150	Column maximum is within expected bounds
ExpectColumnStdevToBeBetween	column='price', min_value=5, max_value=50	Standard deviation is within expected range
ExpectColumnQuantileValuesToBeBetween	column='age', quantile_ranges={...}	Quantile values (e.g. 25th/50th/75th) in ranges
ExpectColumnPairValuesAToBeGreaterThanB	column_A='received_ts', column_B='sent_ts', or_equal=True	Values in column A are greater than (or equal to) values in column B, row by row
ExpectMulticolumnSumToEqual	column_list=['tax', 'discount', 'net_price'], sum_total=100	The sum of row values across a specified list of columns equals a given total for every row

GX Expectations — Schema (Pandas)

Validate table structure: column names, order, count and data types

Expectation Method	Example Argument(s)	Validates What
ExpectColumnToExist	column='user_id'	Specific column is present in the table
ExpectColumnValuesToBeOfType	column='age', type_='int64'	Column dtype matches expected type string
ExpectTableColumnsToMatchOrderedList	column_list=['id','name','email']	Columns present AND in exact order
ExpectTableColumnsToMatchSet	column_set=['id','name'], exact_match=True	Columns present (set match, order not checked)
ExpectTableColumnCountToEqual	value=6	Exact number of columns matches spec
ExpectTableColumnCountToBeBetween	min_value=4, max_value=10	Number of columns is within expected range

GX Expectations — Uniqueness (Pandas)

Detect duplicate records and validate key column uniqueness

Expectation Method	Example Argument(s)	Validates What
ExpectColumnValuesToBeUnique	column='user_id'	Every value in the column is unique (no duplicates)
ExpectCompoundColumnsToBeUnique	column_list=['last_name','phone']	Combination of columns forms a unique composite key
ExpectColumnUniqueValueCountToBeBetween	column='category', min_value=3, max_value=10	Number of distinct values is in expected range
ExpectColumnDistinctValuesToBeInSet	column='country', value_set=[...]	All distinct values are from the allowed set
ExpectColumnDistinctValuesToContainSet	column='country', value_set=['USA','UK']	At minimum these values exist in the column
ExpectColumnDistinctValuesToEqualSet	column='status', value_set=['A','B','C']	Distinct values exactly equal the given set

💡 ExpectColumnValuesToBeUnique is the most common for **primary** keys.

ExpectCompoundColumnsToBeUnique is ideal for **composite** keys (e.g. last_name + phone).

GX Expectations — **Validity** (Pandas)

Validate allowed values, formats, patterns and categorical constraints

Expectation Method	Example Argument(s)	Validates What
ExpectColumnValuesToBeInSet	column='country', value_set=['USA','UK','Canada']	Values belong to allowed categorical set
ExpectColumnValuesToNotBeInSet	column='status', value_set=['DELETED']	Values do NOT appear in a forbidden set
ExpectColumnValuesToMatchRegex	column='email', regex=r'.+@.+'	Values match a regex pattern (e.g. email format)
ExpectColumnValuesToMatchRegexList	column='code', regex_list=[r'\d{3}', r'[A-Z]+']	Values match at least one regex from a list
ExpectColumnValueLengthsToBeBetween	column='name', min_value=2, max_value=100	String length is within expected range
ExpectColumnValueLengthsToEqual	column='country_code', value=2	String length equals exact value (e.g. ISO codes)
ExpectColumnMostCommonValueToBeInSet	column='country', value_set=['USA','UK']	Most frequent value is from expected set

GX Expectations — Volume (Pandas)

Validate row counts, data volume and aggregate quantities

Expectation Method	Example Argument(s)	Validates What
ExpectTableRowCountToEqual	value=200	Exact row count matches expected number
ExpectTableRowCountToBeBetween	min_value=100, max_value=100000	Row count is within acceptable range (not empty, not bloated)



Volume expectations are essential in pipelines to detect truncated loads, missing partitions, or unexpected data explosions before they impact downstream analytics.

GX — Completeness, Uniqueness & Distribution Expectations

```
# COMPLETENESS – required columns must have no NULLs
for col in ['user_id', 'email', 'name']:
    suite.add_expectation(
        gx.expectations.ExpectColumnValuesToNotBeNull(column=col)
    )

# UNIQUENESS – key columns must have no duplicates
for col in ['user_id', 'email']:
    suite.add_expectation(
        gx.expectations.ExpectColumnValuesToBeUnique(column=col)
    )

# DISTRIBUTION – statistical sanity checks
suite.add_expectation(
    gx.expectations.ExpectColumnMeanToBeBetween(column='age', min_value=30, max_value=60)
)
suite.add_expectation(
    gx.expectations.ExpectTableRowCountToBeBetween(min_value=100, max_value=100000)
)
```



Pro Tip: Use loops to apply the same expectation to multiple columns — avoids repetition.

GX Steps 4 & 5 — ValidationDefinition & Running

Step 4: Create a ValidationDefinition (links batch + suite)

```
validation_def = context.validation_definitions.add(  
    gx.ValidationDefinition(  
        name='users_validation', # human-readable name  
        data=batch_def,         # which data to validate  
        suite=suite              # which rules to apply  
    )  
)
```

Step 5: Execute the validation

```
results = validation_def.run(batch_parameters={'dataframe': df})
```

results.success → True if ALL expectations passed, False otherwise

results.results → list of individual ExpectationValidationResult objects

Optional: generate HTML Data Docs (local browser report)

```
context.build_data_docs()
```

```
context.open_data_docs() # opens browser with HTML report
```



context.build_data_docs() writes HTML to a local folder. context.open_data_docs() opens it in your browser.

results.success → bool (True = all pass) | **results.results** → list of result objects per expectation

Reading GX Results — `_print_report()` Explained

```
def _print_report(results):
    success = results.success # bool: True if ALL expectations passed

    for exp_result in results.results: # iterate each expectation
        exp_type = exp_result.expectation_config.type # e.g.
        'ExpectColumnValuesToNotBeNull'
        col      = exp_result.expectation_config.kwargs.get('column', 'table-level')
        passed   = exp_result.success # bool for this single expectation

        if not passed and exp_result.result:
            r = exp_result.result
            if r.get('unexpected_count'):
                print(f'    Issues : {r["unexpected_count"]} unexpected values')
            if r.get('partial_unexpected_list'):
                print(f'    Sample : {r["partial_unexpected_list"][:3]}')
```

Result object attributes:

`.expectation_config.type` = expectation class name | `.expectation_config.kwargs` = rule parameters | `.result` = stats dict



`partial_unexpected_list[:3]` shows only the first 3 bad values — useful to not clutter terminal output.

GX Validation — Output JSON Structure

`vdef.run(batch_parameters={'dataframe': df})` returns a `ValidationResult` object serializable to JSON

```
{
  "success": false,
  "statistics": {
    "evaluated_expectations": 5,
    "successful_expectations": 3,
    "unsuccessful_expectations": 2,
    "success_percent": 60.0
  },
  "results": [
    {
      "expectation_config": {
        "type": "expect_column_values_to_not_be_null",
        "kwargs": {"column": "email", "mostly": 0.99}
      },
      "success": false,
      "result": {
        "element_count": 200,
        "unexpected_count": 1,
        "unexpected_percent": 0.5,
        "partial_unexpected_list": [null]
      }
    },
    ...
  ]
}
```

```
{
  "expectation_config": {
    "type": "expect_column_values_to_be_between",
    "kwargs": {"column": "age", "min_value": 0,
    "max_value": 120}
  },
  "success": false,
  "result": {
    "element_count": 200,
    "unexpected_count": 1,
    "unexpected_percent": 0.5,
    "partial_unexpected_list": [-5]
  }
},
...
]
```

💡 `results[i].result.partial_unexpected_list` shows sample bad values. Use `result_format='COMPLETE'` to get all unexpected values.

GX Data Docs — Auto-Generated HTML Reports

Data Docs are GX's killer feature — automatically generated, shareable HTML reports showing:

- All expectations and whether they passed or failed
- **Sample of unexpected values that caused failures**
- Profiling statistics (count, min, max, mean, missing%) per column
- Full history of all validation runs (with timestamps)

```
context.build_data_docs()    # generate HTML files in local GX directory
context.open_data_docs()    # open in default browser

# With ephemeral context, docs are in a temp directory.
# With file-based context (mode='file'), docs saved in:
# ./gx/uncommitted/data_docs/local_site/index.html
```

⚠ Data Docs won't persist with mode='ephemeral'. Use **mode='file'** for persistent docs in production.

★ Pro Tip: Share Data Docs with stakeholders by hosting the HTML folder on a static web server or S3.

GX Data Docs — HTML Output: Suites & Validation Results

`context.build_data_docs()` generates; `context.open_data_docs()` opens in browser

After calling `context.build_data_docs()`, GX generates a static HTML site at:
`great_expectations/uncommitted/data_docs/local_site/index.html`

Key pages in the HTML Data Docs site:

Overview (`index.html`)

- Lists all Expectation Suites and Validation Results
- Shows pass/fail status with color coding (green = pass, red = fail)
- Displays run timestamps and overall success rate

Suite Detail Page (`expectations/users_suite.html`)

- Lists every Expectation with description
- Shows expected values and constraints
- Column-level details with example values

Validation Result Page (`validations/.../result.html`)

- Per-expectation PASS / FAIL status
- `unexpected_count`, `unexpected_percent` for each failure
- Sample unexpected values (`partial_unexpected_list`)
- Statistics: `evaluated`, `successful`, `failed`, `success_percent`

Open in browser: `context.open_data_docs()` # opens `index.html` automatically

Great Expectations — Tips, Pitfalls & Pros/Cons

- Pros**
- 100+ built-in expectations — no custom code needed
 - Auto Data Docs — professional HTML reports
 - Industry standard — widely used in data engineering
 - Scales to Spark and SQL — not just pandas

- Cons**
- Heavier dependency (~many packages)
 - Steeper learning curve — more concepts
 - v1.x API changed significantly from 0.x
 - Slower to set up for small one-off scripts

Common Pitfalls

- ⚠ Using 0.x tutorials with v1.x code → Check docs version at greatexpectations.io — API changed completely
- ⚠ `mode='ephemeral'` loses Data Docs → Use `mode='file'` when you need persistent reports
- ⚠ Forgetting `batch_parameters` in `run()` → Without passing `{'dataframe': df}`, GX can't find the data
- ⚠ Not handling the context close → In production, wrap in `try/finally` or use context manager

Section 5: Pandas vs Great Expectations

— Comparison

Pandas vs Great Expectations — Feature Comparison

Feature	Pure Pandas	Great Expectations v1.x
Dependencies	pandas only	great-expectations + many
Setup time	~5 minutes	~15-30 minutes
Learning curve	Low (beginner-friendly)	Medium-High
Built-in checks	You write all code	100+ pre-built expectations
HTML reports	None (print only)	Auto Data Docs
Custom logic	Easy — pure Python	Possible but less flexible
Team sharing	Share rules as Python dicts	ExpectationSuite JSON files
Scales to Big Data	Pandas memory limit	Yes (Spark, SQL)
Best for	Learning, small scripts	Production pipelines, teams

When to Use Each — Decision Guide

Use Pure Pandas when...

- You are learning — focus on pandas concepts first
- It's a one-off script or quick EDA notebook
- Your team doesn't use GX yet
- You need highly custom validation logic
- Minimal dependencies are a hard requirement

Use Great Expectations when...

- Building production data pipelines
- Working in a team — share suites as JSON
- You need HTML reports for stakeholders
- Validating data at scale (Spark / SQL)
- You want a data contract standard

Recommendation: Learn Pandas first to understand the concepts, then graduate to GX for real projects.



Pro Tip: In many projects, you'll use both: GX for standard pipeline checks, and custom pandas for edge-case business rules.

Code Comparison — Same Check in Both Versions

Checking age is between 0 and 120:

Pure Pandas

```
range_rules = {'age': {'min': 0, 'max': 120}}

# df[condition] = boolean filter
below = df[df['age'] < 0]
above = df[df['age'] > 120]

if len(below) > 0:
    issues.append(
        f'{len(below)} ages below 0'
    )
```

Great Expectations

```
suite.add_expectation(
    gx.expectations
        .ExpectColumnValuesToBeBetween(
            column='age',
            min_value=0,
            max_value=120
        )
)
# GX handles the check,
# reporting, and HTML docs
```



Both approaches detect the same error (age=-5 at row 10). GX requires less code and gives richer output.

Quick Reference Cheat Sheet

Pure Pandas

```
# Install
pip install pandas numpy

# Load
df = pd.read_csv('f.csv', parse_dates=['d'])

# Validate
v = DataValidator()
v.validate_schema(df, cols, types)
v.validate_completeness(df, req_cols)
v.validate_ranges(df, range_rules)
v.validate_uniqueness(df, unique_cols)
v.validate_categorical(df, cat_rules)
v.generate_report()

# Key pandas ops
df[col].isnull().sum()    # count NaN
df[col].duplicated().sum() # count dups
df[df[col] < 0]           # filter bad
set(vals) - set(allowed)  # find invalid
str(df[col].dtype)        # get type str
```

Great Expectations v1.x

```
# Install
pip install great-expectations pandas

# Setup
context = gx.get_context(mode='ephemeral')
src = context.data_sources.add_pandas('src')
asset = src.add_dataframe_asset('asset')
bdef =
asset.add_batch_definition_whole_dataframe('b')
suite =
context.suites.add(gx.ExpectationSuite('suite'))

# Add rules
suite.add_expectation(gx.expectations
    .ExpectColumnValuesToBeBetween(col, min, max))
suite.add_expectation(gx.expectations
    .ExpectColumnValuesToNotBeNull(col))

# Run
vdef = context.validation_definitions.add(
    gx.ValidationDefinition('v', bdef, suite))
results = vdef.run({'dataframe': df})
context.build_data_docs()
```